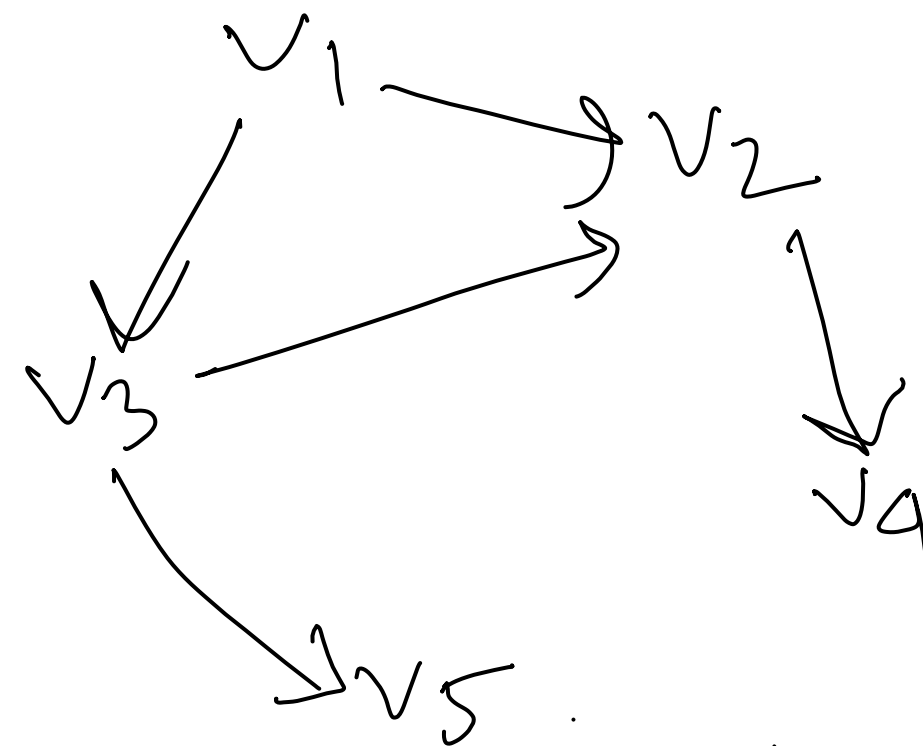
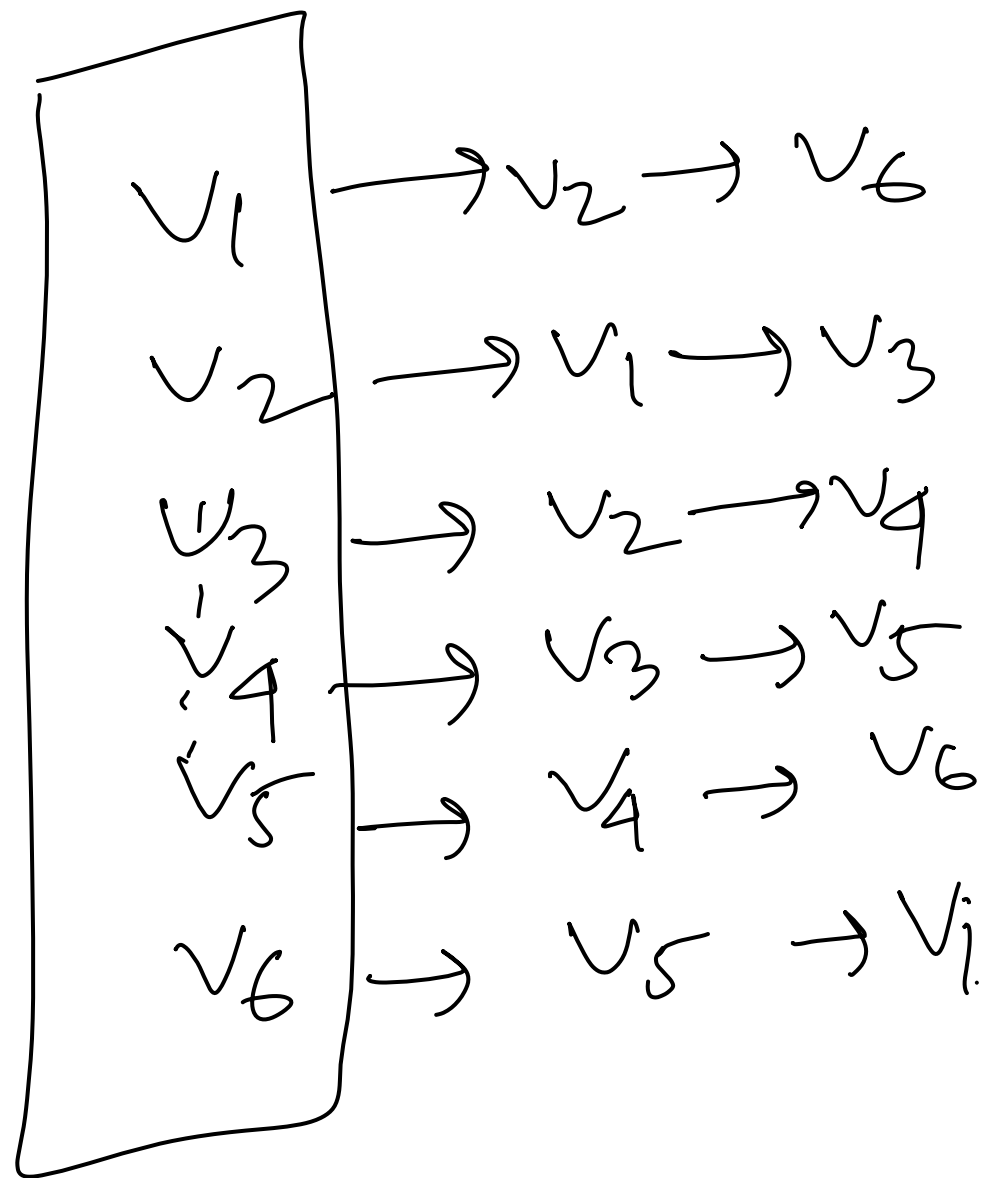




$v_1 \rightarrow v_2 \rightarrow v_3$

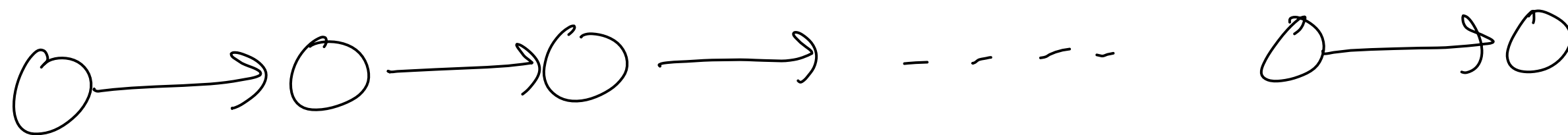
 $v_2 \rightarrow v_4$

 $v_3 \rightarrow v_5$

 ~~$v_4 \rightarrow v_3$~~

 v_5

Linked List



Each list item

— key

— next

Operations

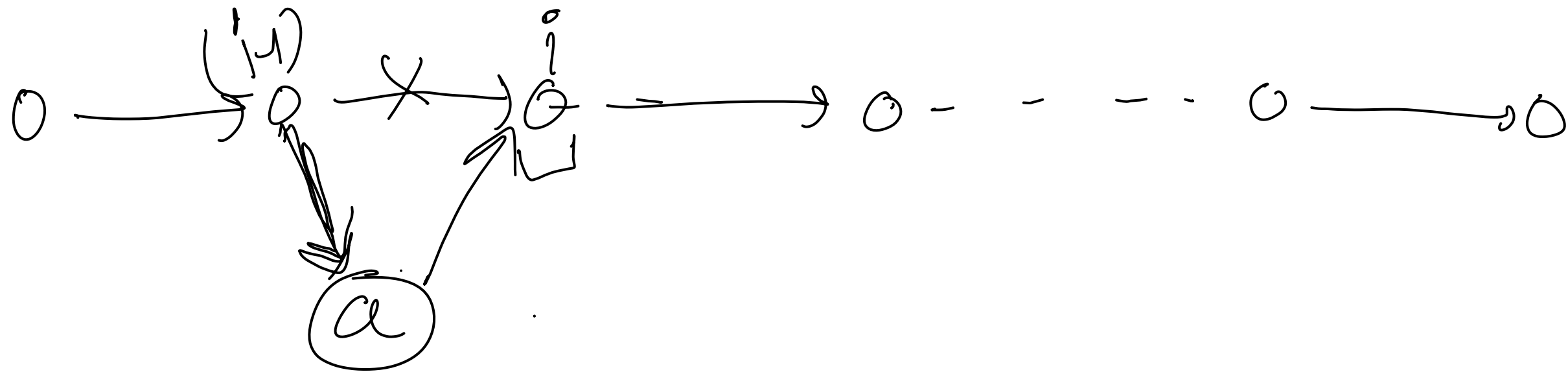
- $Create(a) :$ creates a singleton list with key a
- $Insert(L, i, a) :$ insert a at i -th posⁿ
- $Delete(L, i) :$ remove i -th item from L
- $Search(L, a) :$ yes/no, depending on $a \in L$ or not

```

Procedure Create (a)
  X ← empty linked list item
  X.key ← a
  X.next ← NULL
  return X

```

(-3)



Procedure Insert (L, i, a)

if ($i = 1$)

$X \leftarrow \text{Create}(a)$

$X.\text{next} \leftarrow L$

return X

$\text{temp} \leftarrow L$

while ($i > 2$) if ($\text{temp} \neq \text{NULL}$) return error.

$\text{temp} \leftarrow \text{temp}.\text{next}$

$i--$

endwhile

~~$\text{prev} \leftarrow \text{temp}$~~

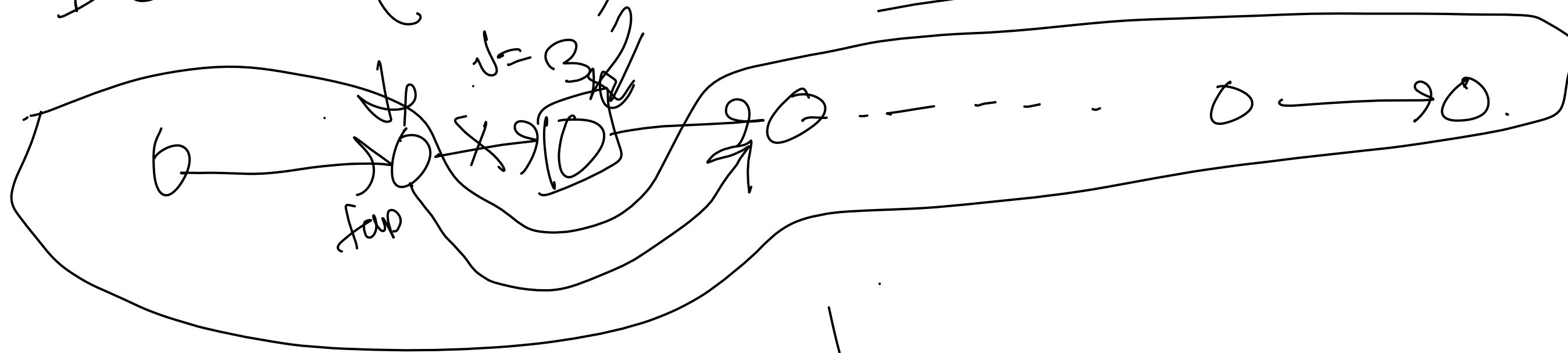
$X \leftarrow \text{Create}(a)$

$\text{temp}.\text{next} \leftarrow X$

```
x.next ← prev.  
return L.
```

Delete (L, i)

free ()



skipped, very similar to meerkat

Procedure Search (L, a)

temp $\leftarrow$ L

while (temp $\neq$ NULL)

if (temp.key = a) return yes end if

temp $\leftarrow$ temp.~~to~~next

end while.

return no.

Adj. list vs adj. matrix

Facebook Graphs!

Adj list storage is much more efficient.

$\theta(n^2)$, $\theta(n + m)$
sparse graph $\downarrow$
edges.

$n = \text{billion}$

hypothetical example

edge queries lots of

$(1, 5) \rightarrow \text{yes}$
 $\rightarrow \text{no.}$

$\theta(1)$, dense graph.

$\theta(l)$, $l = |L|$ length of vertex index list

nbrs

Q: How many neighbors v_i has got.

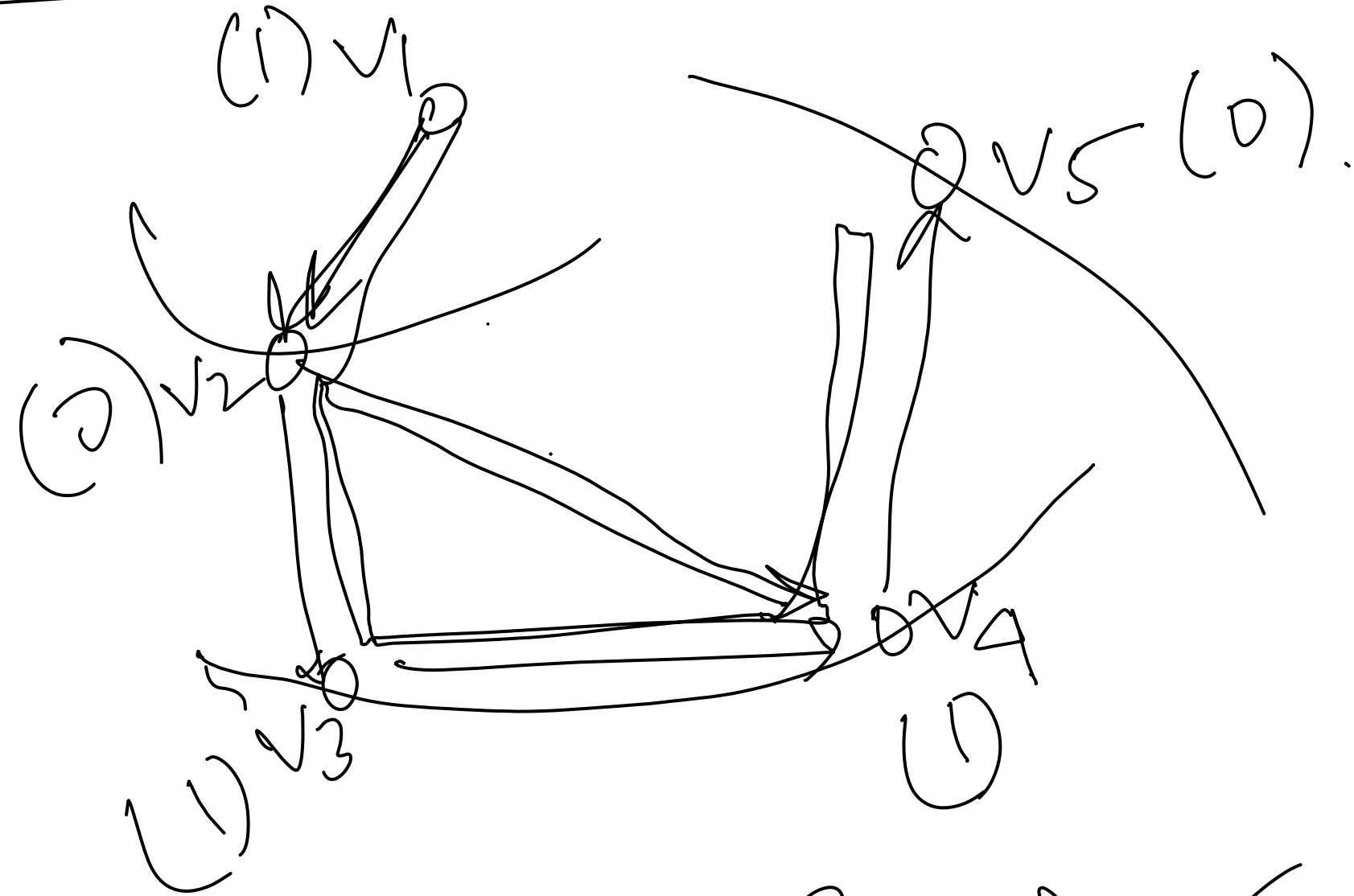
Adj matrix : $\theta(n)$

Adj list : $\theta(l)$

(Sparse graph)

, $l = \# \text{ neighbors of } v_i$
 $(i, \underline{j})$

Graph Search / Exploration :



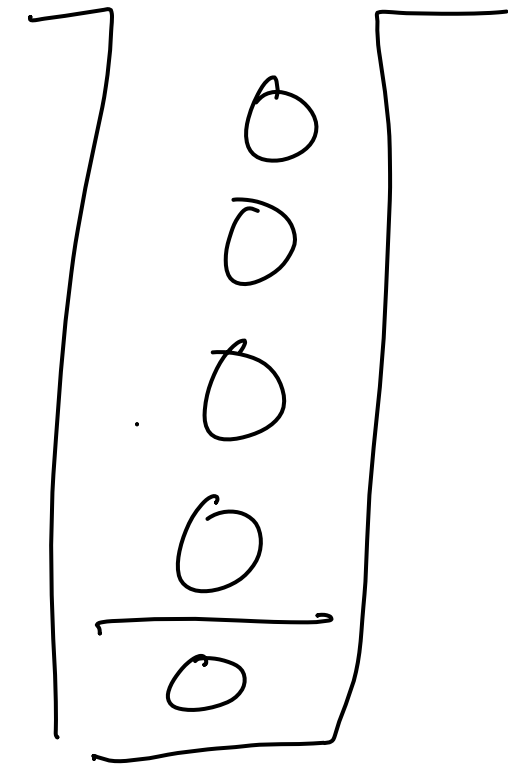
- depth-first-search (DFS) ✓ → Stack
- breadth-first-search (BFS) ✓ → Queue.

Stack Data Structure

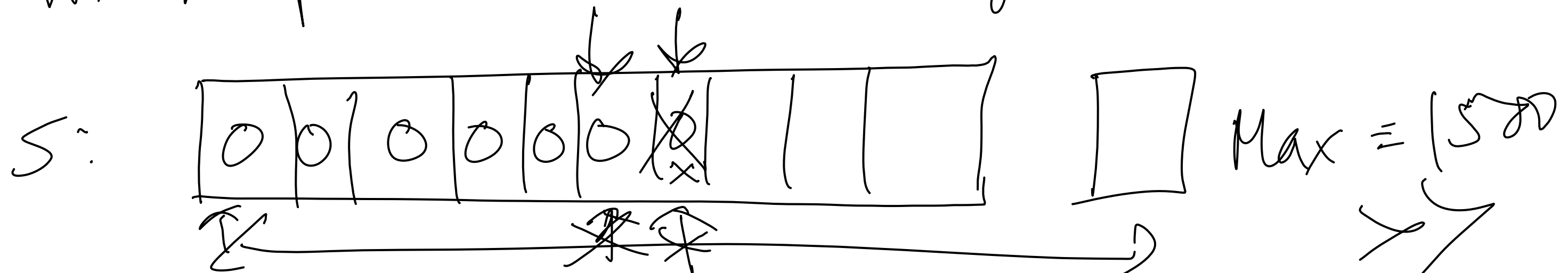
$S_{top} \leftarrow$ index of the top item.

$Push(S, X) \leftarrow$ pushes item X on top of stack S .

$Pop(S) \leftarrow$ removes the top item of S .



we'll implement stack using an array



Initially for an empty Stack $S.top \leftarrow 0$

Procedure CreateStack ()

~~A~~ $S \leftarrow$ empty array of size MAX

$S.top \leftarrow 0$

Return S .

Procedure Push (S, x)
 $S[S.top + 1] \leftarrow x$
 $S.top \leftarrow S.top + 1$
~~if (S.top > MAX) return error.~~
return error.

Procedure Pop (S).
if (S.top = 0)
return error.

$S.top \leftarrow S.top - 1$
return $S[S.top + 1]$

Depth first Search (DFS)

Given an adjacency list, explore the graph.

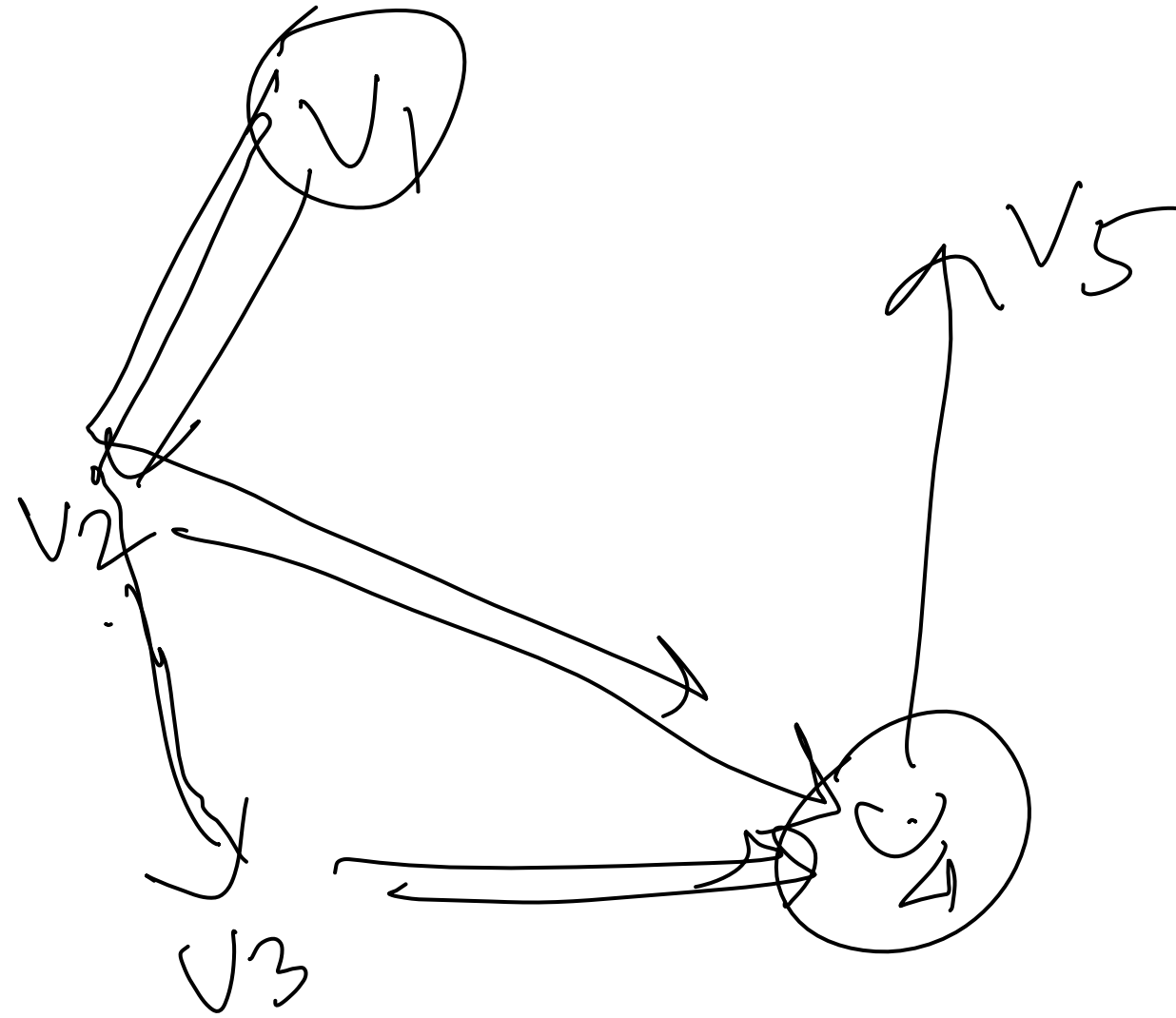
~~Main~~ Count $f(\text{nodes})$.

Q:

Q:

reachability : Is node v_j reachable
from node v_i ?

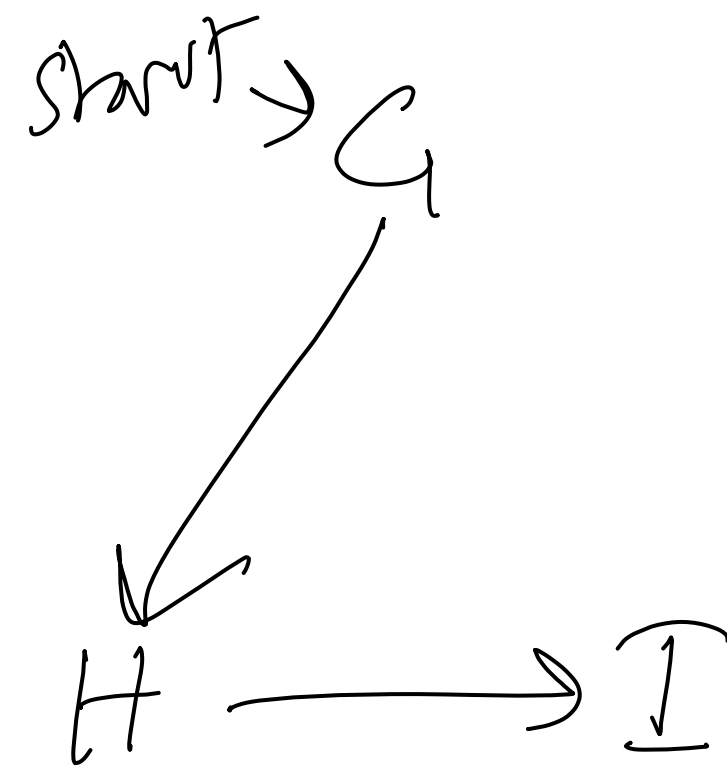
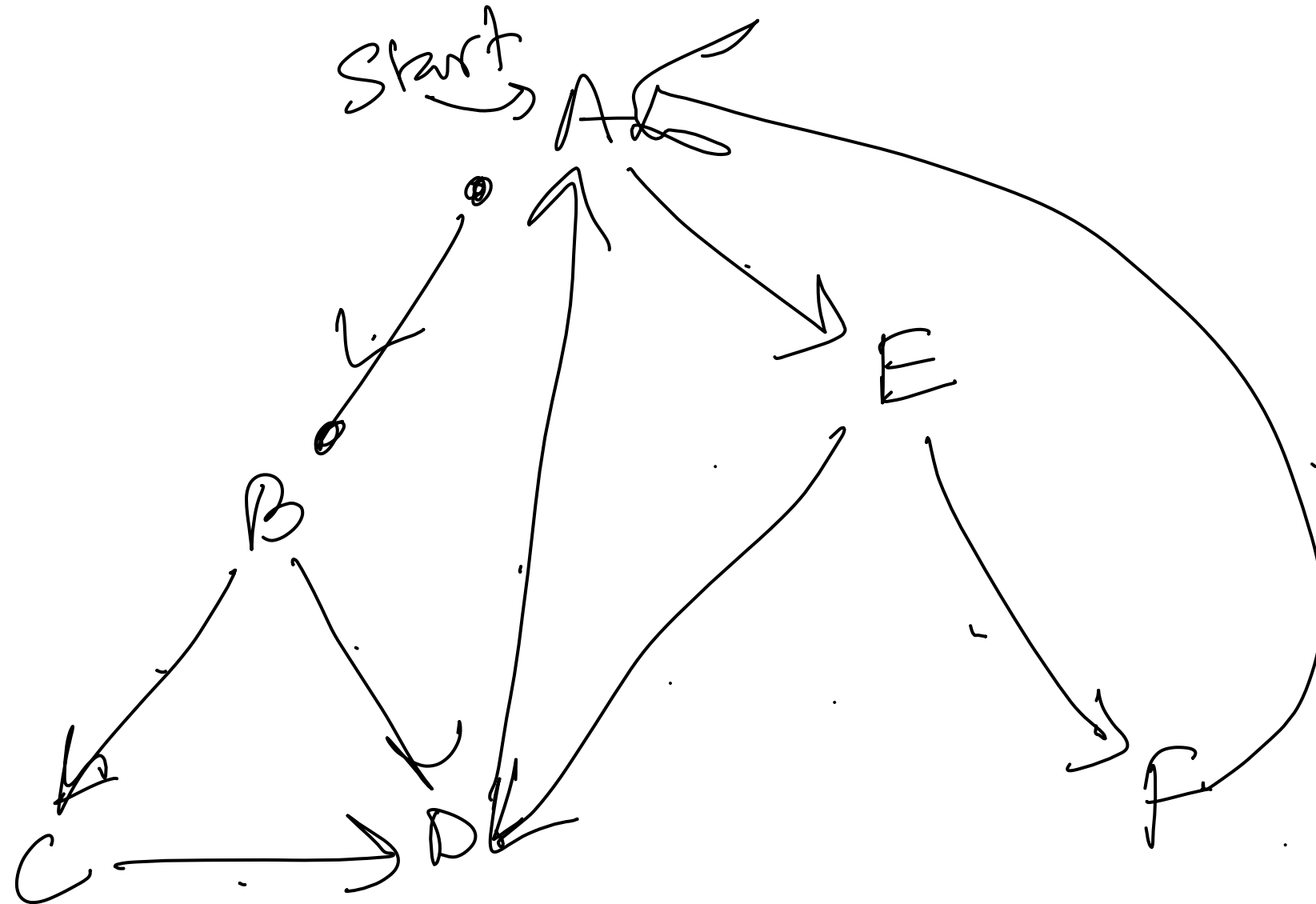
Yes/no.



Is V_4 reachable from V_1 ? yes.

— Yes, two paths.

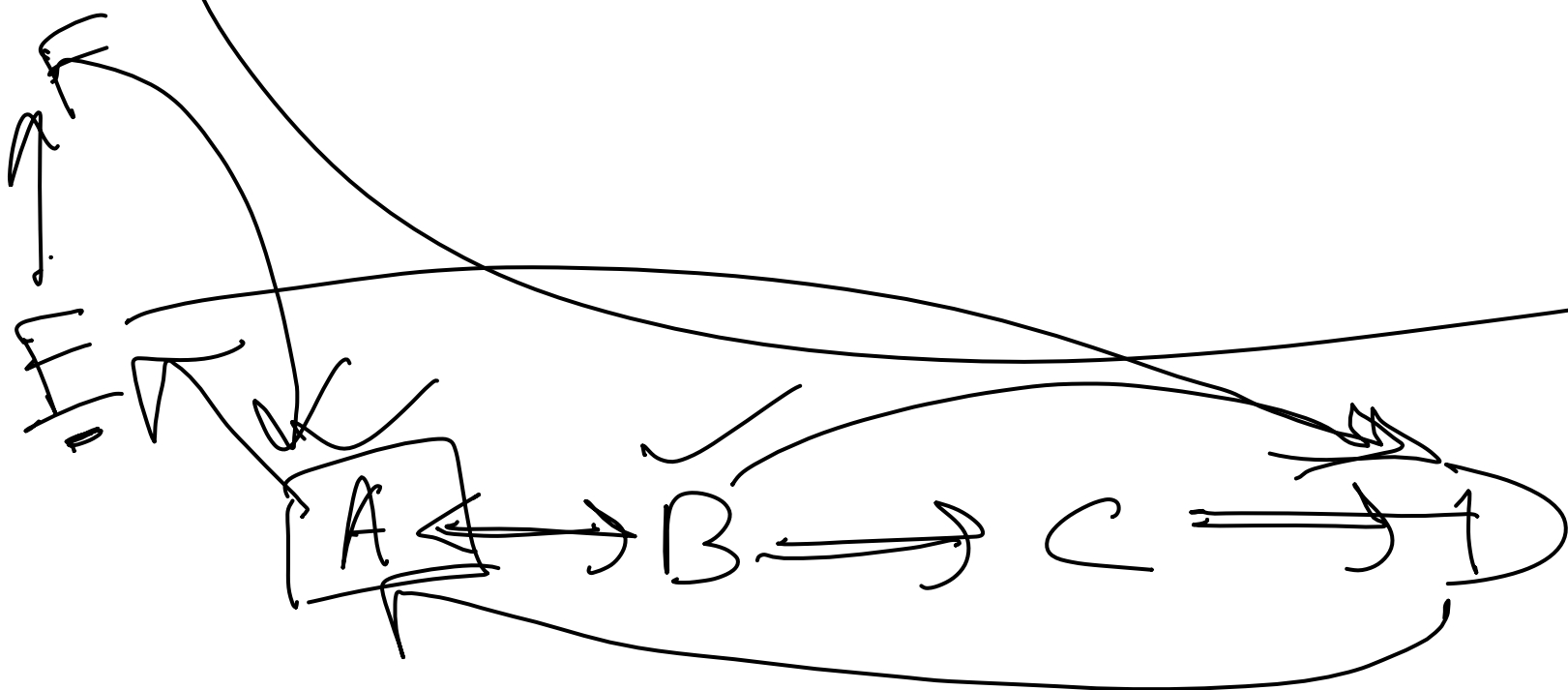
Is V_1 reachable from V_5 ? no.
 — $\nexists$ any path.



n = 9

m = 11

A: [A] [B] [C] [D] [E] [H] [I]



Procedure DFS(G, v, A)

(V, E)
↓
vertices edges.

Starting vertex

A is of size n .

$A[i] = 1$ iff vertex
 i has been explored.

Stack $\leftarrow$ CreateStack().

~~Visit for~~

for each $v \in V$.

$A[v] \leftarrow 0$.

16.08.2024

